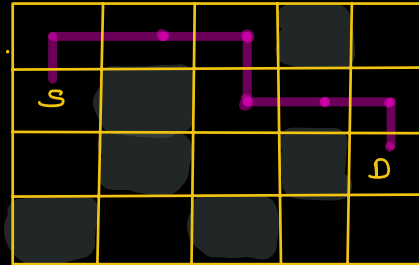
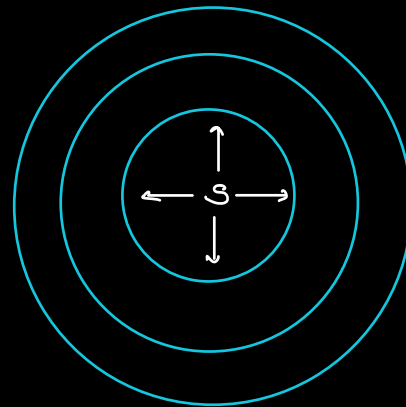


Q. Give a $N \times M$ grid with a source & a destination.
Find the length of shortest path from the source to the destination.

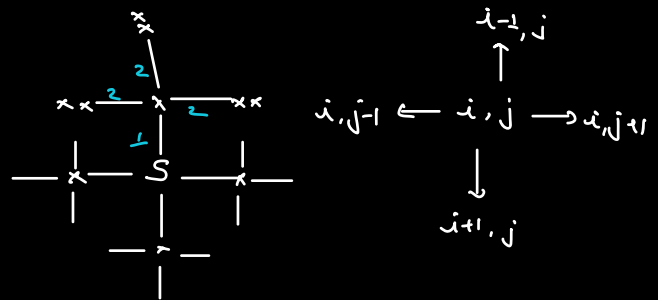


7

	0	1	2	3	4
0	1	2	3		7
1	S 0		4	5	6
2	1	2	3		D 7
3		3			



BFS



~~$\{(1,0), 0\}$~~ $\{(0,0), 1\}$ $\{(2,0), 1\}$

for visited info {
Set
visited matrix $\text{int}[][]$
visited matrix $\text{boolean}[][]$
use the same matrix $\times$

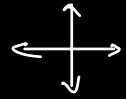
TC: $O(NM)$

```
class NodeData {
    int dist;
    Coordinates c;
} // const
```

```
class Coordinates {
    int i;
    int j;
} // const
```

Amazon Q Given a 2D matrix having locations of houses & hospitals.

For every house find the min distance from a hospital.



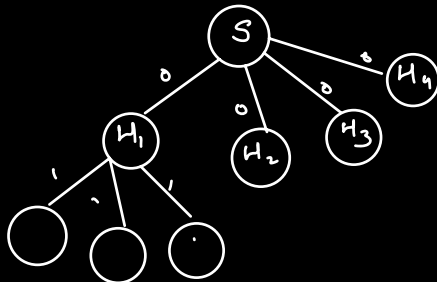
Adobe
Flipkart
Samsung
...
Scaler

R	R	R	H
R	R	H	H
R	H	H	R

3	2	1	0
2	1	0	0
1	0	0	1

A*
Multi threaded

3	2	1	0
2	1	0	0
1	0	0	1



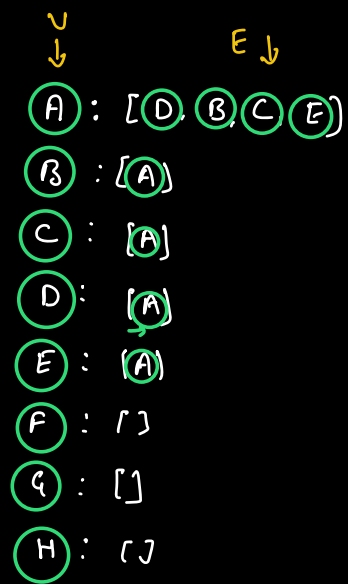
	0	1	2	3
0	R	R	R	H
1	R	R	H	H
2	R	H	H	R

3	2	1	0
2	1	0	0
1	0	0	1

~~{(0,3), 0}~~ ~~{(1,2), 0}~~ ~~{(1,3), 0}~~ ~~{(2,1), 0}~~ ~~{(2,2), 0}~~ ~~{(0,2), 1}~~ ~~{(1,1), 1}~~ ~~{(2,3), 1}~~

~~{(0,0), 1}~~ ~~{(0,1), 2}~~ ~~{(1,0), 2}~~ ~~{(0,0), 3}~~

TC: O(NM)



BFS $\rightarrow O(V+E)$

[A]

[D]

[A, C, E, G]

[D, G, H, F]

[E]

[D, E]

[E]

Amazon
Facebook
MS
AirBnb
hired In
:

Rotting Oranges



Given a 2D grid.

0 → Empty cell

1 → Fresh orange

2 → Rotten orange



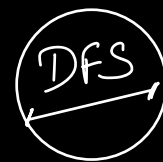
Every minute any fresh orange that is adjacent to a rotten orange becomes rotten.

Return min no of minutes after which all oranges become rotten. Return -1 if not possible.

2	1	1
1	1	0
0	1	1

↓
1

← 2



BFS Template

① Create queue & visited matrix

Q $\longrightarrow$ Node data

visited[N][M] $\longrightarrow$ {false}

② Insert source in Q & mark it visited

NodeData S = new NodeData(Si, Sj, 0);

Q.enqueue(S);

visited[Si][Sj] = true;

③ Keep popping from Q & keep adding the adj nodes to the queue until queue is empty.

while (Q.size() > 0) {

node = Q.dequeue();

i = node.c.i;

j = node.c.j;

i-1, j

i, j-1

i, j+1

i+1, j

row: [-1, 0, 0, 1]

col: [0, -1, 1, 0]

for (K=0; K<4; K++) {

if (i+row[K] >= 0 && i+row[K] < N

&& j+col[K] >= 0 && j+col[K] < M

&& visited[i+row[K]][j+col[K]] == false

&& mat[i+row[K]][j+col[K]] == 0) {

// check for the current cell begins

// insert in queue

// mark visited

}

}